Lab Assignment 5: Generative AI

Name	Krushna Prashant Warule	**Dept**	Computer
Roll No	_____	**Div**	B
Subject	Advance Data Science	**Assignment No**	5
GitHub	https://github.com/krushnawarule/Advance_Data_Science_Lab_SCOE/tree/main/Assignment_5_Generative_AI		

1. Aim

To implement a Generative AI application using a language model and understand how language models generate meaningful and contextually relevant text from user-provided prompts.

2. Theory

A language model estimates **P(next word | previous words)**. Text generation repeats one loop: take the prompt, compute the probability of every possible next token, **sample** one token, append it to the context and repeat until the sentence ends. **Temperature** controls randomness (low = safe and repetitive, high = diverse), and **top-k** restricts sampling to the k most likely tokens. Large Language Models such as GPT use the same loop, but compute the probabilities with a very large Transformer trained on huge text data.

Simulation used: no internet access or LLM weights were available, so a small **trigram language model** is 'pre-trained' on a corpus of 300 data-science sentences by counting word triples. It is used for prompt-based generation with temperature and top-k sampling. This reproduces the mechanism of an LLM at a very small scale; it does not have the knowledge or fluency of a real LLM. With internet access the same prompts can be sent to a pre-trained model such as GPT-2 through the Hugging Face pipeline('text-generation') interface.

3. Dataset

Simulated text corpus of 300 sentences about data science and machine learning (3,368 words, 62-word vocabulary), saved as `dataset.csv`.

4. Code (core part; full code in code.py)

```
counts = defaultdict(Counter)                          # "pre-training": learn P(next word | last 2 words)
for line in corpus:
    w = ["<s>", "<s>"] + line.split()
    for i in range(2, len(w)): counts[(w[i - 2], w[i - 1])][w[i]] += 1

def next_probs(ctx, temp=1.0, top_k=None):
    c = counts.get(tuple(ctx[-2:])) or counts[("<s>", "<s>")]
    words = list(c); p = np.array([c[x] for x in words], float) ** (1 / temp)   # temperature
    if top_k:                                                                    # top-k filtering
        p[np.argsort(p)[:-top_k]] = 0
    return words, p / p.sum()

def generate(prompt, temp=1.0, top_k=None, max_words=14, seed=0):
    rng = np.random.RandomState(seed); out = prompt.split()
    ctx = ["<s>", "<s>"] + out                          # separate list (not the same object as out)
    for _ in range(max_words):
        words, p = next_probs(ctx, temp, top_k); nxt = rng.choice(words, p=p)
        if nxt == ".": break
        out.append(nxt); ctx.append(nxt)
    return " ".join(out) + " ."
```

5. Output

Console output (output.txt):

```
Corpus: 300 sentences, 3368 words, vocabulary=62
Learned contexts (trigram states): 179

--- Prompt-based generation (temperature = 0.7, top-k = 5) ---
Prompt: 'data science'
  -> data science helps to analyse hidden patterns using training data .
Prompt: 'machine learning'
  -> machine learning is used to predict accurate predictions across many domains .
Prompt: 'a neural network'
  -> a neural network is used to predict accurate predictions across many domains .
Prompt: 'a data scientist'
  -> a data scientist helps to analyse hidden patterns using training data .
Prompt: 'the model'
  -> the model is used to predict accurate predictions across many domains .
```

```
--- Effect of temperature on the prompt 'a neural network' ---
T=0.3: a neural network often requires useful features for better decision making .
T=1.0: a neural network often requires useful features for better decision making .
T=2.0: a neural network often requires useful features using training data .

Next-word probabilities after 'the model':
  T=0.5: can=0.36, needs=0.28, is=0.21, often=0.11
  T=1.0: can=0.28, needs=0.25, is=0.22, often=0.16
  T=2.0: can=0.24, needs=0.23, is=0.21, often=0.18

Perplexity on 2 unseen sentences: 2.83
```

Output images:

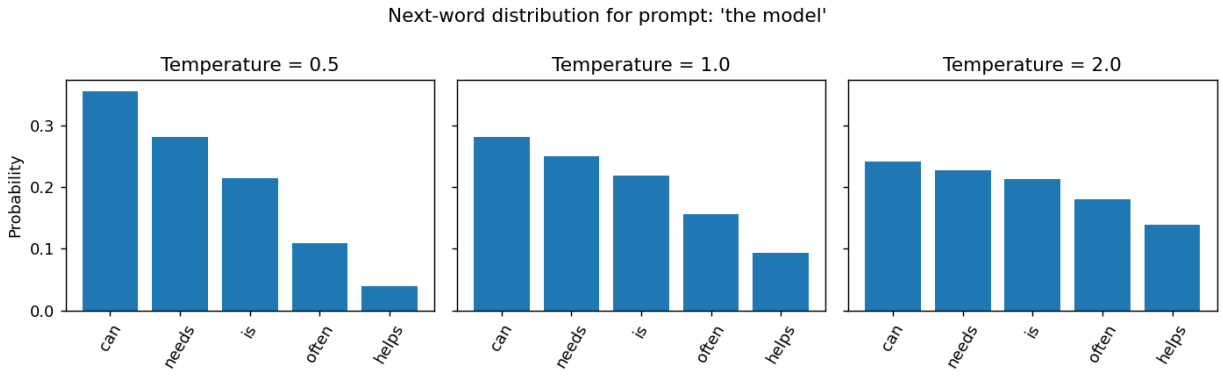

Fig 1: Next-word probability distribution after the prompt 'the model' at three temperatures

6. Conclusion

The model generated grammatical, topic-related sentences from different prompts (for example 'data science helps to analyse hidden patterns using training data'). The probability charts show the effect of temperature: a low temperature (0.5) sharpens the distribution towards the most likely word, while a high temperature (2.0) flattens it so that more varied words can be chosen. The trigram model only recombines phrases it saw during training and cannot reason or create new knowledge, whereas a real LLM uses a Transformer with billions of parameters to give far more fluent and flexible answers. Prompt wording and sampling settings strongly influence the generated text.

7. References

[1] T. Brown et al., 'Language Models are Few-Shot Learners', NeurIPS 2020.

[2] A. Vaswani et al., 'Attention Is All You Need', NeurIPS 2017.

[3] D. Jurafsky, J. Martin, Speech and Language Processing, 3rd ed. draft (Chapter 3: N-gram Language Models).

[4] Hugging Face documentation: 'Text generation with Transformers', huggingface.co/docs.